

osl-base-extended — Developer Documentation

Version: 0.1.12

Author: Bilal Raza

Angular: 21.2.0+

License: ISC

Table of Contents

1. [Overview](#)
 2. [Installation & Setup](#)
 3. [Architecture](#)
 4. [API Layer — Httpbase](#)
 5. [Base UI Service — baseComponent](#)
 6. [CRUD UI — osl-setup](#)
 7. [Dynamic Forms — DynamicForm](#)
 8. [Data Grid — OslGrid](#)
 9. [Inline Form Grid — OslFormGrid](#)
 10. [Form Components Reference](#)
 11. [Skeleton Loading — OslSkeletonDirective](#)
 12. [Datasource Cache Service](#)
 13. [Utility Namespaces](#)
 14. [Full TypeScript Interface Reference](#)
 15. [Advanced Patterns](#)
 16. [Troubleshooting](#)
-

1. Overview

`osl-base-extended` is an **enterprise Angular library** that eliminates repetitive boilerplate when building data-driven applications. It provides:

- A fully-typed **HTTP abstraction layer** with authentication, CRUD shorthands, and error normalization
- A **zero-config CRUD page component** (`<osl-setup>`) that combines grid, dialog, forms, pagination, and delete confirmation in one tag
- A **declarative form engine** that generates any form from a JSON configuration array
- A **GPU-accelerated skeleton loading directive** with 8 preset layouts and 4 animation modes
- **200+ pure utility functions** across 7 namespaces — array, date, number, object, string, storage, and validation

The library targets teams that need to ship enterprise screens fast without sacrificing type safety or flexibility.

2. Installation & Setup

Install the package

```
npm install osl-base-extended
```

Install peer dependencies

```
npm install @angular/material @angular/cdk
```

Configure the HTTP client

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptorsFromDi()),
  ],
};
```

Import library modules

```
// app.module.ts
import { FormStructureModule, OslSkeletonModule } from 'osl-base-extended';

@NgModule({
  imports: [
    FormStructureModule, // includes all form components + osl-setup + osl-grid
    OslSkeletonModule,   // includes [oslSkeleton] directive
  ],
})
export class AppModule {}
```

Set the authentication token

The library reads the JWT from `localStorage` automatically on every request:

```
// After login
localStorage.setItem('token', response.token);

// On logout
localStorage.removeItem('token');
```

3. Architecture

```

osl-base-extended
├── core/
│   ├── base/          baseComponent      ← UI helpers (snackbar, dialog)
│   └── http/          Httpbase           ← HTTP abstraction layer
├── shared/
│   ├── components/
│   │   ├── dialog-wrapper/  DialogWrapper  ← Material dialog container
│   │   └── delete-confirmation/ DeleteConfirmation
│   ├── directives/
│   │   └── skeleton/      OslSkeletonDirective  ← Skeleton loading
│   │                           OslSkeletonThemeService
│   └── form-structure/
│       ├── setup/        OslSetup          ← Full CRUD page
│       ├── grid/         OslGrid           ← Data table
│       ├── form-grid/    OslFormGrid       ← Inline editable grid
│       ├── dynamic-form/ DynamicForm       ← Declarative form engine
│       ├── datasource-cache/ DatasourceCacheService
│       ├── input/        Oslinput
│       ├── textarea/     Osltextarea
│       ├── select/       OslSelect
│       ├── autocomplete/ OslAutocomplete
│       ├── autocomplete-listener/ OslAutocompleterListener
│       ├── radio/        OslRadio
│       ├── checkbox/     OslCheckbox
│       ├── slide-toggle/ OslSlideToggle
│       ├── datepicker/   OslDatepicker
│       ├── file-upload/  OslFileUpload
│       ├── button/       OslButton
│       └── searchbar/    OslSearchbar
└── util/
    ├── ArrayUtil
    ├── DateUtil
    ├── NumberUtil
    ├── ObjectUtil
    ├── StringUtil
    ├── StorageUtil
    └── ValidationUtil
  
```

4. API Layer — Httpbase

Concept

Httpbase is an **abstract class**. Create one concrete service per backend controller by extending it and passing the controller name to `super()`. You immediately inherit all HTTP verb wrappers, CRUD shorthands, auth injection, and error normalization.

```
/api/{controllerName}/{methodName}?{params}
```

The base URL is `/api/` — configure a proxy in `angular.json` or `proxy.conf.json` to forward requests to your backend.

Creating a service

```
import { Injectable } from '@angular/core';
import { Httpbase } from 'osl-base-extended';

@Injectable({ providedIn: 'root' })
export class ProductService extends Httpbase {
  constructor() {
    super('Product'); // all requests go to /api/Product/...
  }
}
```

Built-in public CRUD methods

These are ready to call from any component without any additional code in the service.

`getAll<T>()`

```
// GET /api/Product/GetAll
const res = await this.productSvc.getAll<Product[]>();
if (res.isSuccessful) {
  this.products = res.result;
}
```

`getById<T>(id)`

```
// GET /api/Product/GetById?id=42
const res = await this.productSvc.getById<Product>(42);
```

`save<T>(body)`

```
// POST /api/Product/Save {body as JSON}
const res = await this.productSvc.save<Product>({ name: 'Widget', price: 9.99 });
```

```
if (res.isSuccessful) {  
    this.base.showSuccess('Product created!');  
}
```

update<T>(body)

```
// PUT /api/Product/Update {body as JSON}  
const res = await this.productSvc.update<Product>({ id: 42, name: 'Widget Pro' });
```

remove<T>(id)

```
// DELETE /api/Product/Delete?id=42  
const res = await this.productSvc.remove(42);
```

search(body)

```
// POST /api/Product/Search {body as JSON}  
const res = await this.productSvc.search({ category: 'Electronics', page: 1,  
    pageSize: 25 });
```

getConfig()

```
// GET /api/Product/getConfig  
const res = await this.productSvc.getConfig();  
// Used internally by OslAutocompleteLister to get grid column definitions
```

Custom endpoints using protected verb wrappers

Override in your service class to expose additional endpoints.

```
@Injectable({ providedIn: 'root' })  
export class ProductService extends Httpbase {  
    constructor() { super('Product'); }  
  
    // POST with custom method name  
    async bulkImport(products: Product[]) {  
        return this.post<ImportResult>('BulkImport', { products });  
    }  
  
    // PUT  
    async updateStock(id: number, quantity: number) {
```

```

    return this.put<Product>('UpdateStock', { id, quantity });
  }

  // PATCH – partial update
  async setFeatured(id: number, featured: boolean) {
    return this.patch<Product>('SetFeatured', { id, featured });
  }

  // DELETE with multiple query params
  async archiveMany(ids: number[]) {
    return this.delete<void>('ArchiveMany', [
      { property: 'ids', value: ids } // sends ?ids=1&ids=2&ids=3
    ]);
  }

  // GET with query params
  async getCategory(categoryId: number, page: number) {
    return this.get<Product[]>('GetByCategory', [
      { property: 'categoryId', value: categoryId },
      { property: 'page', value: page },
    ]);
  }

  // File upload (multipart/form-data, 60s timeout)
  async uploadImage(productId: number, file: File) {
    const fd = new FormData();
    fd.append('productId', String(productId));
    fd.append('image', file);
    return this.upload<{ imageUrl: string }>('UploadImage', fd);
  }
}

```

Protected verb wrappers reference

Method signature	HTTP	Timeout	Notes
<code>post<T>(method, body)</code>	POST	30 s	JSON body, auto-sets Content-Type
<code>get<T>(method, params?)</code>	GET	30 s	<code>myParams[]</code> → query string
<code>put<T>(method, body)</code>	PUT	30 s	Full replace
<code>patch<T>(method, body)</code>	PATCH	30 s	Partial update
<code>delete<T>(method, params?)</code>	DELETE	30 s	<code>myParams[]</code> → query string
<code>upload<T>(method, formData)</code>	POST multipart	60 s	Never set Content-Type manually

`myParams` — query parameter helper

```

interface myParams {
  property: string; // query param name
}

```

```

    value: any;          // string, number, or array
  }

  // Array values are expanded: ids=[1,2,3] → ?ids=1&ids=2&ids=3
  this.get('Filter', [
    { property: 'status', value: [1, 2] },
    { property: 'page', value: 1 },
  ]);
  // → GET /api/Controller/Filter?status=1&status=2&page=1

```

HttpResponse<T> — universal response wrapper

Every method returns this, whether the call succeeded or failed:

```

interface HttpResponse<T> {
  isSuccessfull: boolean; // true when statusCode is 2xx
  statusCode: number;      // raw HTTP status code
  error: string;           // human-readable message on failure
  result: T;               // parsed JSON body
  headers?: HttpHeaders;  // raw response headers (useful for pagination totals)
}

```

Error codes handled automatically

Status	Message produced
0	Connection Error! Please Contact Administration
400	error.error.message or "Bad Request"
401	Unauthorized Access
403	You don't have rights to perform this action
404	Resource not found
409	error.error.message or "Conflict"
422	error.error.message or "Validation failed"
500	An error has occurred, Please contact support

Reading response headers

Useful for server-side pagination — backend can send `X-Total-Count` or similar:

```

const res = await this.productSvc.search(query);
const total = Number(res.headers?.get('recordCount') ?? 0);
this.totalRecords = total;

```

Proxy configuration (development)

```
// proxy.conf.json
{
  "/api": {
    "target": "https://your-backend.com",
    "secure": true,
    "changeOrigin": true
  }
}
```

```
// angular.json – under serve > options
"proxyConfig": "proxy.conf.json"
```

5. Base UI Service — baseComponent

Injectable service that provides common UI operations. Inject it in any component.

```
import { baseComponent } from 'osl-base-extended';

@Component({ ... })
export class MyComponent {
  private base = inject(baseComponent);
}
```

Methods

showSuccess(message: string)

Displays a green Material snackbar.

```
this.base.showSuccess('Record saved successfully.');
```

showError(error: string)

Displays a red Material snackbar.

```
this.base.showError(res.error);
```

navigate(url: string)

Navigates using Angular Router.

```
this.base.navigate('/dashboard');
```

`openDialog(header, formBody, formFooter, width, data?, component?)`

Opens a Material dialog wrapped in `DialogWrapper`.

```
this.base.openDialog(  
  'Edit Profile',          // dialog title  
  ProfileFormComponent,   // component rendered as form body  
  ProfileFooterComponent, // component rendered as footer (save/cancel buttons)  
  '60vw',                 // dialog width  
  { userId: this.userId } // data injected into dialog components  
);
```

`openDeleteDialog(message, title, confirmText, cancelText, data?)`

Opens the pre-built delete confirmation dialog. Returns a dialog ref — subscribe to `afterClosed()` for the result.

```
this.base.openDeleteDialog(  
  `Are you sure you want to delete "${item.name}"?`,  
  'Confirm Delete',  
  'Yes, Delete',  
  'Cancel',  
  item  
);
```

6. CRUD UI — osl-setup

The flagship feature. Replaces a full feature module with a single component declaration.

What it renders automatically:

- Page heading with an "Add" button
- Search bar (debounced 300ms)
- Data grid with edit and delete action buttons
- Paginated footer
- Add/Edit dialog with the dynamic form
- Delete confirmation dialog

Minimal example

```
// product.component.ts
@Component({
  template: `
    <osl-setup
      title="Products"
      [columns]="columns"
      [datasource]="products"
      [formElements]="form"
      [loading]="loading"
      [isPaginated]="true"
      [totalRecords]="total"
      (onSave)="onSave($event)"
      (onDelete)="onDelete($event)"
      (onSearch)="onSearch($event)"
      (pageChange)="loadPage($event)"
    />
  `
})
export class ProductComponent {
  private svc = inject(ProductService);
  private base = inject(baseComponent);

  loading = false;
  products: Product[] = [];
  total = 0;
  page = 1;

  columns: OslGridColumn[] = [
    { key: 'name', label: 'Product Name' },
    { key: 'price', label: 'Price', displayFn: (r) => `$$${r.price.toFixed(2)}` },
    { key: 'category', label: 'Category' },
    { key: 'status', label: 'Status', enums: { 1: 'Active', 0: 'Inactive' } },
  ];

  form: elements[] = [
    { key: 'name', label: 'Product Name', elementType: 'textbox', columns: 12, required: true },
    { key: 'price', label: 'Price', elementType: 'textbox', columns: 6, inputType: 'number' },
    { key: 'categoryId', label: 'Category', elementType: 'select', columns: 6,
      datasource: this.categories, displayField: 'name', valueField: 'id' },
    { key: 'status', label: 'Status', elementType: 'slide-toggle', columns: 6 },
    { key: 'description', label: 'Description', elementType: 'textarea', columns: 12, textareaRows: 4 },
  ];

  async ngOnInit() { await this.load(); }

  async load() {
```

```
this.loading = true;
const res = await this.svc.search({ page: this.page, pageSize: 25 });
this.loading = false;
if (res.isSuccessful) {
  this.products = res.result;
  this.total = Number(res.headers?.get('recordCount') ?? 0);
}
}

async onSave(e: { model: Product; mode: 'add' | 'edit' }) {
  const res = e.mode === 'add'
    ? await this.svc.save(e.model)
    : await this.svc.update(e.model);
  if (res.isSuccessful) {
    this.base.showSuccess(e.mode === 'add' ? 'Product added!' : 'Product
updated!');
    await this.load();
  } else {
    this.base.showError(res.error);
  }
}

async onDelete(product: Product) {
  const res = await this.svc.remove(product.id);
  if (res.isSuccessful) {
    this.base.showSuccess('Product deleted.');
```

```
    await this.load();
  }
}

async onSearch(q: string) {
  // re-query with search term
}

async loadPage(e: OslPageEvent) {
  this.page = e.page;
  await this.load();
}
}
```

All inputs

Input	Type	Default	Description
title	string	—	Entity label shown in heading and dialog title
columns	OslGridColumn[]	—	Table column definitions
datasource	any[]	—	Rows bound to the table
formElements	elements[]	—	Form configuration for add/edit dialog
loading	boolean	false	Triggers skeleton rows in the grid

Input	Type	Default	Description
<code>isPaginated</code>	<code>boolean</code>	<code>false</code>	Shows pagination footer
<code>pageSize</code>	<code>number</code>	<code>25</code>	Initial page size
<code>totalRecords</code>	<code>number</code>	—	Total row count for server-side pagination
<code>autoMode</code>	<code>boolean</code>	<code>true</code>	Library handles client-side sort and pagination internally
<code>tableHeight</code>	<code>string</code>	—	CSS height for scrollable table body (e.g. <code>'500px'</code>)
<code>dialogWidth</code>	<code>string</code>	<code>'50vw'</code>	Width of the add/edit Material dialog
<code>isLister</code>	<code>boolean</code>	<code>false</code>	Hides the edit/delete actions column
<code>beforeDisplay</code>	<code>(model: any) => any</code>	—	Transform the row object before the dialog opens
<code>onAddEditFn</code>	<code>(model, mode) => void</code>	—	Replaces the built-in dialog with a custom handler

All outputs

Output	Payload type	Fires when
<code>onAdd</code>	—	Add button clicked
<code>onEdit</code>	<code>row any</code>	Edit button on a row clicked
<code>onDelete</code>	<code>row any</code>	Delete confirmed in the confirmation dialog
<code>onSave</code>	<code>{ model: any; mode: 'add' 'edit' }</code>	Save clicked inside the dialog
<code>onSearch</code>	<code>string</code>	Search input emits (300ms debounce)
<code>pageChange</code>	<code>OslPageEvent</code>	Page number changes
<code>pageSizeChange</code>	<code>number</code>	Page size dropdown changes
<code>sortChange</code>	<code>OslSortEvent</code>	Column header clicked for sorting
<code>onRowClick</code>	<code>row any</code>	Any table row is clicked

7. Dynamic Forms — DynamicForm

Generates complete, validated, reactive forms from a declarative configuration array. No template code needed beyond the component tag.

```
<osl-dynamic-form
  [elements]="formConfig"
  [(model)]="formData"
  [skeletonLoading]="loading"
/>
```

The `model` input is two-way bound — changes inside the form propagate out automatically.

`elements` interface — complete reference

```
interface elements {
  // — Required —————
  key:      string;           // model property name
  label:    string;           // field label
  elementType: ElementType;   // field type (see below)
  columns:  number;           // 1-12 grid columns

  // — Visibility & state —————
  disabled?: boolean;
  hide?:    boolean;
  disabledIf?: () => boolean;
  hideIf?:    (model: any) => boolean;
  loadingIf?: (model: any) => boolean; // shows spinner on select/autocomplete

  // — Validation —————
  required?: boolean;
  requiredIf?: (model: any) => boolean;

  // — Events —————
  change?: (model: any) => void; // fires on field value change

  // — Datasource (select, radio, autocomplete) ———
  datasource?: any[];
  displayField?: string;
  valueField?:  string;

  // — API-backed datasource —————
  apiService?: any;           // injected service instance
  apiMethod?:  string;        // method name on service
  apiConfigMethod?: string;
  apiBody?:    any;

  // — textbox options —————
  inputType?: InputType;       // 'text' | 'password' | 'email' | 'number'
  | 'tel' | 'url'
  placeholder?: string;
  mask?:        string;        // e.g. '(000) 000-0000'
  min?:         string | number;
  max?:         string | number;
  minLength?:   number;
```

```

maxLength?:    number;
prefixIcon?:   string;           // Material icon name
suffixIcon?:   string;

// — textarea options —————
textareaRows?:    number;
characterCounter?: boolean;
resize?:          TextareaResize; // 'none' | 'both' | 'horizontal' |
'vertical'

// — select options —————
selectPlaceholder?: string;
clearable?:         boolean;

// — datepicker options —————
dateType?: DateInputType; // 'date' | 'datetime-local' | 'time' | 'month' |
'week'
minDate?:    string;
maxDate?:    string;
inline?:     boolean;

// — radio / checkbox options —————
labelPosition?: 'before' | 'after';
indeterminate?: boolean;

// — slide-toggle options —————
trueLabel?:    string;
falseLabel?:   string;

// — file upload options —————
accept?:       string; // e.g. 'image/*,.pdf'
multiple?:     boolean;
maxFileSize?:  number; // bytes

// — autocomplete options —————
autocompletePlaceholder?: string;
searchType?:             'Api' | 'Local';
objectName?:             string;

// — fieldset —————
rows?: elements[]; // nested child elements

// — templateRef —————
templateRef?: TemplateRef<any>;
}

```

Element type reference

elementType	Component rendered	Key options
'textbox'	<osl-input>	inputType, mask, prefixIcon, suffixIcon, min/max, minLength/maxLength
'textarea'	<osl-textarea>	textareaRows, characterCounter, resize
'select'	<osl-select>	datasource, displayField, valueField, clearable, apiService
'autocomplete'	<osl-autocomplete>	searchType, datasource, apiService, apiMethod, objectName
'radio'	<osl-radio>	datasource, displayField, valueField, labelPosition
'checkbox'	<osl-checkbox>	labelPosition, indeterminate
'slide-toggle'	<osl-slide-toggle>	trueLabel, falseLabel, labelPosition
'datepicker'	<osl-datepicker>	dateType, minDate, maxDate, inline
'file-uploader'	<osl-file-upload>	accept, multiple, maxFileSize
'button'	<osl-button>	change (click handler)
'fieldset'	Container	rows (nested elements[])
'templateRef'	Custom content	templateRef

Conditional logic example

```
formElements: elements[] = [
  {
    key: 'accountType',
    label: 'Account Type',
    elementType: 'select',
    columns: 6,
    datasource: [
      { id: 'personal', name: 'Personal' },
      { id: 'business', name: 'Business' },
    ],
    displayField: 'name',
    valueField: 'id',
  },
  {
    key: 'companyName',
    label: 'Company Name',
    elementType: 'textbox',
    columns: 6,
    hideIf: (m) => m.accountType !== 'business',
    requiredIf: (m) => m.accountType === 'business',
  }
]
```

```

    },
    {
      key: 'taxId',
      label: 'Tax ID',
      elementType: 'textbox',
      columns: 6,
      hideIf: (m) => m.accountType !== 'business',
      disabledIf: () => !this.hasPermission('TAX_EDIT'),
    },
  ],
];

```

Nested fieldsets

```

{
  key: 'address',
  label: 'Address',
  elementType: 'fieldset',
  columns: 12,
  rows: [
    { key: 'street', label: 'Street', elementType: 'textbox', columns: 12 },
    { key: 'city', label: 'City', elementType: 'textbox', columns: 6 },
    { key: 'country', label: 'Country', elementType: 'select', columns: 6,
      datasource: this.countries, displayField: 'name', valueField: 'code' },
  ],
}

```

API-backed select datasource

The datasource is fetched once and cached via [DatasourceCacheService](#):

```

{
  key: 'departmentId',
  label: 'Department',
  elementType: 'select',
  columns: 6,
  apiService: inject(DepartmentService), // must extend Httpbase
  apiMethod: 'getAll', // called as service.getAll()
  displayField: 'name',
  valueField: 'id',
}

```

8. Data Grid — OslGrid

A standalone table component used inside `<osl-setup>` but also available independently.

```
<osl-grid
  [columns]="columns"
  [datasource]="rows"
  [isPaginated]="true"
  [totalRecords]="total"
  [loading]="loading"
  tableHeight="450px"
  (editClick)="onEdit($event)"
  (deleteClick)="onDelete($event)"
  (pageChange)="onPage($event)"
  (sortChange)="onSort($event)"
  (onRowClick)="onRowSelect($event)"
/>
```

OslGridColumn interface

```
interface OslGridColumn {
  key:      string;           // model property path
  label:    string;           // column header
  enums?:   Record<any, string>; // value map: { 1: 'Active', 0:
'Inactive' }
  displayFn?: (row: any) => string; // custom cell renderer
  isActions?: boolean;           // marks the edit/delete column (auto-
generated)
}
```

Inputs

Input	Type	Default	Description
columns	OslGridColumn[]	—	Column definitions
datasource	any[]	—	Row data
isPaginated	boolean	false	Show pagination footer
pageSize	number	25	Rows per page
totalRecords	number	—	Total count (server-side)
autoMode	boolean	true	Library handles pagination/sort internally
tableHeight	string	—	CSS height for scrollable body
loading	boolean	false	Show skeleton rows
isSelectable	boolean	false	Enable row selection mode

Outputs

Output	Type	Description
<code>editClick</code>	<code>row</code>	Edit button clicked
<code>deleteClick</code>	<code>row</code>	Delete button clicked
<code>pageChange</code>	<code>OslPageEvent</code>	Page number changes
<code>pageSizeChange</code>	<code>number</code>	Page size dropdown changes
<code>sortChange</code>	<code>OslSortEvent</code>	Column sort toggled
<code>onRowClick</code>	<code>row</code>	Row clicked

Enum display example

```
columns: OslGridColumn[] = [  
  {  
    key: 'status',  
    label: 'Status',  
    enums: {  
      0: 'Draft',  
      1: 'Active',  
      2: 'Archived',  
    }  
  }  
];  
// Cell shows 'Active' when row.status === 1
```

9. Inline Form Grid — OslFormGrid

A data grid where cells contain editable form elements.

```
<osl-form-grid  
  [columns]="gridColumns"  
  [datasource]="lineItems"  
  (rowAdd)="addRow()"  
  (rowDelete)="deleteRow($event)"  
>
```

OslFormGridColumn interface

```
interface OslFormGridColumn {  
  key:      string;  
  label:    string;  
  formElem?: elements;  // if present, cell is editable using this config  
}
```

10. Form Components Reference

All form components use the same two-way binding pattern:

```
<osl-input [(model)]="user.name" label="Name" [required]="true" />
```

OslButton

```
<osl-button  
  label="Save"  
  variant="primary"  
  size="md"  
  [loading]="saving"  
  [disabled]="form.invalid"  
  [fullWidth]="false"  
  (click)="save()"  
>
```

Variants: `primary` | `secondary` | `success` | `danger` | `warning` | `info` | `outline-primary` | `outline-secondary` | `icon`

Sizes: `sm` | `md` | `lg`

OslInput (textbox)

```
<osl-input  
  [(model)]="value"  
  label="Phone"  
  inputType="tel"  
  mask="(000) 000-0000"  
  prefixIcon="phone"  
  [maxLength]="15"  
  [required]="true"  
>
```

OslSelect

```
<osl-select
  [(model)]="form.status"
  label="Status"
  [datasource]="statuses"
  displayField="name"
  valueField="id"
  [clearable]="true"
/>
```

OslAutocomplete

```
<!-- Local search -->
<osl-autocomplete
  [(model)]="form.city"
  label="City"
  searchType="Local"
  [datasource]="cities"
  displayField="name"
  valueField="id"
/>

<!-- API search (calls service.search() + opens lister dialog) -->
<osl-autocomplete
  [(model)]="form.customerId"
  label="Customer"
  searchType="Api"
  [apiService]="customerSvc"
  displayField="fullName"
  valueField="id"
  objectName="Customer"
/>
```

OslDatepicker

```
<osl-datepicker
  [(model)]="form.startDate"
  label="Start Date"
  dateType="date"
  minDate="2024-01-01"
  maxDate="2030-12-31"
/>
```

Date types: `date` | `datetime-local` | `time` | `month` | `week`

OslFileUpload

```
<osl-file-upload
  [(model)]="form.attachments"
  label="Attachments"
  accept=".pdf,.docx,image/*"
  [multiple]="true"
  [maxFileSize]="5242880"
/>
```

OslSlideToggle

```
<osl-slide-toggle
  [(model)]="form.isActive"
  label="Active"
  trueLabel="Yes"
  falseLabel="No"
/>
```

OslSearchbar

Debounced search input (300ms). Useful for wiring to server-side data:

```
<osl-searchbar (search)="onSearch($event)" placeholder="Search products..." />
```

11. Skeleton Loading — OslSkeletonDirective

Add `[oslSkeleton]="loading"` to any element. The directive overlays the element with animated skeleton bones while loading is `true`.

Import

```
import { OslSkeletonModule } from 'osl-base-extended';
```

Basic usage

```
<!-- Auto-detects child layout -->
<div [oslSkeleton]="isLoading">
  <h2>{{ title }}</h2>
  <p>{{ description }}</p>
</div>
```

All preset types

```
<!-- Text rows -->
<div [oslSkeleton]="loading" oslSkeletonType="text" [oslSkeletonRows]="4"></div>

<!-- Rectangle / image placeholder -->
<div [oslSkeleton]="loading" oslSkeletonType="rect" oslSkeletonMinHeight="200px">
</div>

<!-- Circle avatar -->
<img [oslSkeleton]="loading" oslSkeletonType="circle"
oslSkeletonCircleSize="56px">

<!-- Card (image + lines) -->
<div [oslSkeleton]="loading" oslSkeletonType="card" [oslSkeletonCardLines]="3">
</div>

<!-- List -->
<ul [oslSkeleton]="loading" oslSkeletonType="list" [oslSkeletonListItems]="6">
</ul>

<!-- Table -->
<table [oslSkeleton]="loading" oslSkeletonType="table"
      [oslSkeletonTableRows]="10" [oslSkeletonTableCols]="5">
</table>

<!-- Avatar + text row -->
<div [oslSkeleton]="loading" oslSkeletonType="avatar-text"></div>
```

All inputs

Input	Type	Default	Description
oslSkeleton	boolean	—	Main toggle
oslSkeletonType	string	'auto'	Layout preset
oslSkeletonAnimation	string	'shimmer'	'shimmer' 'pulse' 'wave' 'none'
oslSkeletonTheme	string	'light'	'light' 'dark'
oslSkeletonColor	string	—	Custom bone background color
oslSkeletonHighlight	string	—	Custom shimmer highlight color
oslSkeletonRadius	string	—	Border radius override
oslSkeletonRows	number	3	Row count for type='text'
oslSkeletonRowGap	string	'10px'	Gap between text rows
oslSkeletonDuration	number	1500	Animation duration (ms)

Input	Type	Default	Description
<code>oslSkeletonDelay</code>	number	0	Delay before skeleton appears (ms)
<code>oslSkeletonMinHeight</code>	string	—	Minimum height when host has no size
<code>oslSkeletonCircleSize</code>	string	'40px'	Diameter for <code>type='circle'</code>
<code>oslSkeletonListItems</code>	number	4	Items for <code>type='list'</code>
<code>oslSkeletonTableRows</code>	number	5	Rows for <code>type='table'</code>
<code>oslSkeletonTableCols</code>	number	4	Columns for <code>type='table'</code>
<code>oslSkeletonCardLines</code>	number	3	Text lines for <code>type='card'</code>
<code>oslSkeletonBgColor</code>	string	—	Overlay background color
<code>oslSkeletonZIndex</code>	number	10	Z-index of the overlay

Global theme service

```
import { OslSkeletonThemeService } from 'osl-base-extended';

@Component({ ... })
export class AppComponent {
  private theme = inject(OslSkeletonThemeService);

  ngOnInit() {
    // Sync with user's dark mode preference
    const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
    this.theme.setTheme(prefersDark ? 'dark' : 'light');
  }
}
```

12. Datasource Cache Service

`DatasourceCacheService` caches API responses for form dropdowns. It is used automatically by `DynamicForm` when `apiService` is set on an element, but can also be used manually.

```
import { DatasourceCacheService } from 'osl-base-extended';

@Component({ ... })
export class MyComponent {
  private cache = inject(DatasourceCacheService);

  async loadCategories() {
    // First call: hits API. Subsequent calls: returns cached value.
    const res = await this.cache.load(this.categorySvc, 'getAll');
    this.categories = res.result ?? [];
  }
}
```

```
refreshCategories() {
  // Force a fresh API call next time
  this.cache.invalidate(this.categorySvc, 'getAll');
}

clearAllCaches() {
  this.cache.invalidateAll();
}
}
```

Key feature: If the same (service, method, body) combination is requested while a request is already in flight, the second call waits for the first — no duplicate HTTP requests.

13. Utility Namespaces

All utilities are tree-shakable pure functions with no external dependencies. Import by namespace:

```
import {
  ArrayUtil, DateUtil, NumberUtil, ObjectUtil,
  StringUtil, StorageUtil, ValidationUtil
} from 'osl-base-extended';
```

ArrayUtil — 25+ functions

Function	Signature	Description
chunk	(arr, size)	Split array into chunks of given size
unique	(arr)	Remove primitive duplicates
uniqueBy	(arr, key)	Remove duplicates by object key
flatten	(arr)	One-level deep flatten
flattenDeep	(arr)	Recursively flatten
groupBy	(arr, key)	Group objects by property value
sortBy	(arr, key, dir)	Sort objects by key ascending/descending
filterBy	(arr, key, val)	Filter objects by key equals value
sumBy	(arr, key)	Sum numeric property
minBy	(arr, key)	Minimum by property
maxBy	(arr, key)	Maximum by property
first	(arr)	First element
last	(arr)	Last element

Function	Signature	Description
<code>average</code>	<code>(arr)</code>	Mean of number array
<code>isEmpty</code>	<code>(arr)</code>	True if null or length 0
<code>paginate</code>	<code>(arr, page, size)</code>	Slice page from array
<code>move</code>	<code>(arr, from, to)</code>	Move element to new index
<code>toggle</code>	<code>(arr, item)</code>	Add if absent, remove if present
<code>intersection</code>	<code>(a, b)</code>	Elements in both arrays
<code>difference</code>	<code>(a, b)</code>	Elements in a not in b
<code>toMap</code>	<code>(arr, key)</code>	Array to <code>Map</code> by key
<code>compact</code>	<code>(arr)</code>	Remove falsy values
<code>shuffle</code>	<code>(arr)</code>	Random order (Fisher-Yates)
<code>sample</code>	<code>(arr, n)</code>	n random elements
<code>countBy</code>	<code>(arr, key)</code>	Count occurrences per key value
<code>union</code>	<code>(a, b)</code>	Unique merge of two arrays
<code>zip</code>	<code>(a, b)</code>	Pair elements by index
<code>every</code>	<code>(arr, pred)</code>	All elements satisfy predicate
<code>some</code>	<code>(arr, pred)</code>	Any element satisfies predicate
<code>partition</code>	<code>(arr, pred)</code>	Split into [truthy, falsy]

DateUtil — 40+ functions

Function	Description
<code>now()</code>	Current <code>Date</code>
<code>today()</code>	Today at midnight
<code>formatDate(date, pattern)</code>	Pattern: <code>YYYY, MM, DD, HH, mm, ss</code>
<code>toDateOnly(date)</code>	<code>'YYYY-MM-DD'</code> string
<code>toISOString(date)</code>	ISO 8601 string
<code>parseDate(str)</code>	Parse <code>'YYYY-MM-DD'</code> string
<code>isValidDate(date)</code>	True if valid <code>Date</code> object
<code>addDays</code> / <code>subDays</code>	Add or subtract days
<code>addMonths</code> / <code>subMonths</code>	Add or subtract months
<code>addYears</code> / <code>subYears</code>	Add or subtract years

Function	Description
<code>addHours / subHours</code>	Add or subtract hours
<code>addMinutes / subMinutes</code>	Add or subtract minutes
<code>diffInDays(a, b)</code>	Absolute day difference
<code>diffInMonths(a, b)</code>	Absolute month difference
<code>diffInYears(a, b)</code>	Absolute year difference
<code>diffInMinutes(a, b)</code>	Absolute minute difference
<code>isBefore(a, b)</code>	a is before b
<code>isAfter(a, b)</code>	a is after b
<code>isSameDay(a, b)</code>	Same calendar day
<code>isToday(date)</code>	Is today
<code>isWeekend(date)</code>	Saturday or Sunday
<code>isPast(date)</code>	In the past
<code>isFuture(date)</code>	In the future
<code>inRange(date, start, end)</code>	Within inclusive range
<code>startOfDay / endOfDay</code>	Midnight / 23:59:59
<code>startOfMonth / endOfMonth</code>	First / last day of month
<code>startOfYear / endOfYear</code>	First / last day of year
<code>getAge(birthDate)</code>	Age in years
<code>daysInMonth(date)</code>	Days in the month
<code>isLeapYear(year)</code>	Leap year check
<code>getWeekNumber(date)</code>	ISO week number
<code>nextWorkday(date)</code>	Next Monday–Friday
<code>timeAgo(date)</code>	'3 hours ago' / '2 days ago'

NumberUtil — 30+ functions

Function	Description
<code>round(n, decimals)</code>	Round to N decimal places
<code>floor / ceil</code>	Math.floor / Math.ceil
<code>clamp(n, min, max)</code>	Constrain to range
<code>formatCurrency(n, currency)</code>	'\$1,500.00'

Function	Description
<code>formatNumber(n, decimals)</code>	Locale-formatted number string
<code>abbreviate(n)</code>	<code>1500000</code> → <code>'1.5M'</code>
<code>toOrdinal(n)</code>	<code>3</code> → <code>'3rd'</code>
<code>percentage(part, total)</code>	$(\text{part} / \text{total}) * 100$
<code>sum(arr)</code>	Sum number array
<code>average(arr)</code>	Mean of number array
<code>min / max</code>	Minimum / maximum
<code>random(min, max)</code>	Random float in range
<code>randomInt(min, max)</code>	Random integer in range
<code>inRange(n, min, max)</code>	True if n within range
<code>lerp(a, b, t)</code>	Linear interpolation
<code>isPrime(n)</code>	Primality test
<code>isEven / isOdd</code>	Parity check
<code>isPositive / isNegative</code>	Sign check
<code>isFiniteNumber / isNaNValue / isInteger</code>	Type guards
<code>gcd(a, b)</code>	Greatest common divisor
<code>lcm(a, b)</code>	Lowest common multiple
<code>fibonacci(n)</code>	nth Fibonacci number
<code>toRadians / toDegrees</code>	Angle conversion
<code>safeParseFloat / safeParseInt</code>	Returns <code>null</code> instead of <code>NaN</code>

ObjectUtil — 25+ functions

Function	Description
<code>deepClone(obj)</code>	Recursive clone (JSON-safe)
<code>deepMerge(target, source)</code>	Recursive merge
<code>isPlainObject(val)</code>	True if plain <code>{}</code> object
<code>isEmpty(obj)</code>	True if no own keys
<code>isEqual(a, b)</code>	Deep equality check
<code>hasKey(obj, key)</code>	Own key exists
<code>hasPath(obj, path)</code>	Dot-notation path exists

Function	Description
<code>pick(obj, keys)</code>	New object with only listed keys
<code>omit(obj, keys)</code>	New object without listed keys
<code>filterValues(obj, pred)</code>	Keep entries where predicate is true
<code>mapValues(obj, fn)</code>	Transform all values
<code>diff(a, b)</code>	Keys that differ between objects
<code>flattenObject(obj)</code>	<code>{ 'a.b.c': val }</code>
<code>unflattenObject(obj)</code>	Reverse of flatten
<code>getPath(obj, 'a.b.c')</code>	Safe nested get
<code>setPath(obj, 'a.b.c', val)</code>	Safe nested set (mutates)
<code>toQueryString(obj)</code>	<code>'key=val&key2=val2'</code>
<code>fromQueryString(str)</code>	Parse query string to object
<code>invertObject(obj)</code>	Swap keys and values
<code>sortByKey(obj)</code>	Sort keys alphabetically
<code>size(obj)</code>	Number of own keys

StringUtil — 35+ functions

Function	Description
<code>capitalize(str)</code>	First letter uppercase
<code>titleCase(str)</code>	Each Word Capitalized
<code>camelCase(str)</code>	<code>helloWorldFoo</code>
<code>pascalCase(str)</code>	<code>HelloWorldFoo</code>
<code>snakeCase(str)</code>	<code>hello_world_foo</code>
<code>kebabCase(str)</code>	<code>hello-world-foo</code>
<code>isEmpty</code> / <code>isBlank</code>	Null/empty vs whitespace-only
<code>contains(str, sub)</code>	Substring check
<code>countWords(str)</code>	Word count
<code>isPalindrome(str)</code>	Forward equals reverse
<code>truncate(str, len, suffix?)</code>	Shorten with ellipsis
<code>stripHtml(str)</code>	Remove all tags
<code>escapeHtml(str)</code>	Encode <code><</code> <code>></code> <code>&</code> <code>"</code> <code>'</code>

Function	Description
<code>unescapeHtml(str)</code>	Decode HTML entities
<code>toSlug(str)</code>	URL-safe slug
<code>removeSpecialChars(str)</code>	Keep alphanumeric only
<code>removeWhitespace(str)</code>	Remove all spaces
<code>mask(str, visibleEnd)</code>	'****1234'
<code>initials(str)</code>	'Bilal Raza' → 'BR'
<code>countOccurrences(str, sub)</code>	Count substring occurrences
<code>format(template, ...args)</code>	'Hello {0}!' → 'Hello World!'
<code>repeat(str, n)</code>	Repeat string n times
<code>reverseString(str)</code>	Reverse characters
<code>wordWrap(str, width)</code>	Insert newlines at word boundaries
<code>padStart / padEnd</code>	Left/right pad to length
<code>randomString(len)</code>	Random alphanumeric
<code>toBoolean(str)</code>	'true'/'1'/'yes' → true

StorageUtil

```

// localStorage – persists across sessions
StorageUtil.setLocal('key', value);           // JSON.stringify internally
StorageUtil.getLocal<T>('key');               // JSON.parse – returns null if
missing
StorageUtil.removeLocal('key');
StorageUtil.clearLocal();
StorageUtil.hasLocal('key');
StorageUtil.localKeys();                      // string[]

// sessionStorage – cleared on tab close
StorageUtil.setSession('key', value);
StorageUtil.getSession<T>('key');
StorageUtil.removeSession('key');
StorageUtil.clearSession();
StorageUtil.hasSession('key');
StorageUtil.sessionKeys();

// Cookies
StorageUtil.setCookie('key', 'value', 30); // 30 days expiry
StorageUtil.getCookie('key');
StorageUtil.removeCookie('key');
StorageUtil.getAllCookies();                 // Record<string, string>
StorageUtil.hasCookie('key');

```

ValidationUtil — 40+ functions

```

// Format validators (return boolean)
ValidationUtil.isEmail(str)
ValidationUtil.isUrl(str)
ValidationUtil.isPhone(str)
ValidationUtil.isPostalCode(str)
ValidationUtil.isIPv4(str)
ValidationUtil.isIPv6(str)
ValidationUtil.isMACAddress(str)
ValidationUtil.isHexColor(str)
ValidationUtil.isCreditCard(str)           // Luhn algorithm
ValidationUtil.isJSON(str)
ValidationUtil.isASCII(str)
ValidationUtil.isAlpha(str)
ValidationUtil.isAlphanumeric(str)
ValidationUtil.isNumeric(str)
ValidationUtil.isDecimal(str)

// Required / empty
ValidationUtil.isRequired(val)
ValidationUtil.isEmpty(str)

// Length
ValidationUtil.minLength(str, n)
ValidationUtil.maxLength(str, n)

```

```

ValidationUtil.lengthBetween(str, min, max)

// Number range
ValidationUtil.minValue(n, min)
ValidationUtil.maxValue(n, max)
ValidationUtil.inRange(n, min, max)
ValidationUtil.isPositive(n)
ValidationUtil.isNegative(n)
ValidationUtil.isInteger(n)

// Date
ValidationUtil.isValidDate(date)

// Pattern
ValidationUtil.matchesPattern(str, regex)

// Equality
ValidationUtil.isMatch(a, b)

// Password strength
ValidationUtil.isStrongPassword(str, {
  minLength: 8,
  requireUppercase: true,
  requireLowercase: true,
  requireNumber: true,
  requireSpecial: true,
})
ValidationUtil.passwordStrength(str) // 0 (very weak) → 5 (very strong)

```

14. Full TypeScript Interface Reference

```

// — HTTP —

interface HttpResponse<T = any> {
  isSuccessful: boolean;
  statusCode: number;
  error: string;
  result: T;
  headers?: HttpHeaders;
}

interface myParams {
  property: string;
  value: any;
}

// — Grid —

interface OslGridColumn {
  key: string;
}

```

```

    label:      string;
    enums?:    Record<any, string>;
    displayFn?: (row: any) => string;
    isActions?: boolean;
  }

  interface OslPageEvent {
    page:      number;
    pageSize:  number;
  }

  interface OslSortEvent {
    key:      string;
    direction: 'asc' | 'desc';
  }

  // — Form Grid —

  interface OslFormGridColumn {
    key:      string;
    label:    string;
    formElem?: elements;
  }

  interface OslFormGridRowEvent {
    row:    any;
    index:  number;
  }

  // — Dynamic Form —

  type ElementType =
    | 'button' | 'checkbox' | 'textbox' | 'textarea' | 'radio'
    | 'select' | 'datepicker' | 'file-uploader' | 'autocomplete'
    | 'slide-toggle' | 'fieldset' | 'templateRef';

  type InputType      = 'text' | 'password' | 'email' | 'number' | 'tel' | 'url';
  type DateInputType  = 'date' | 'datetime-local' | 'time' | 'month' | 'week';
  type TextareaResize = 'none' | 'both' | 'horizontal' | 'vertical';

  interface elements {
    key:      string;
    label:    string;
    elementType: ElementType;
    columns:  number;
    disabled?: boolean;
    hide?:    boolean;
    required?: boolean;
    disabledIf?: () => boolean;
    hideIf?:    (model: any) => boolean;
    requiredIf?: (model: any) => boolean;
    loadingIf?:  (model: any) => boolean;
    change?:    (model: any) => void;
    datasource?: any[];
  }

```

```

displayField?: string;
valueField?: string;
apiService?: any;
apiMethod?: string;
apiConfigMethod?: string;
apiBody?: any;
inputType?: InputType;
placeholder?: string;
mask?: string;
min?: string | number;
max?: string | number;
minLength?: number;
maxLength?: number;
prefixIcon?: string;
suffixIcon?: string;
textareaRows?: number;
characterCounter?: boolean;
resize?: TextareaResize;
selectPlaceholder?: string;
clearable?: boolean;
dateType?: DateInputType;
minDate?: string;
maxDate?: string;
inline?: boolean;
labelPosition?: 'before' | 'after';
trueLabel?: string;
falseLabel?: string;
accept?: string;
multiple?: boolean;
maxFileSize?: number;
indeterminate?: boolean;
autocompletePlaceholder?: string;
templateRef?: TemplateRef<any>;
searchType?: 'Api' | 'Local';
objectName?: string;
rows?: elements[];
}

```

```
// — Button —
```

```

type ButtonVariant =
  | 'primary' | 'secondary' | 'success' | 'danger' | 'warning' | 'info'
  | 'outline-primary' | 'outline-secondary' | 'icon';

```

```
type ButtonSize = 'sm' | 'md' | 'lg';
```

```
// — Dialog —
```

```

interface Dialog {
  header: string;
  formBody: any;
  formFooter?: any;
  width: string;
  data?: any;
}

```

```
component?: any;
dialogRef?: MatDialogRef<any>;
}

interface DeleteConfirmationData {
  title: string;
  message: string;
  confirmText: string;
  cancelText: string;
  data?: any;
}

// — Password —

interface PasswordOptions {
  minLength?: number; // default 8
  requireUppercase?: boolean;
  requireLowercase?: boolean;
  requireNumber?: boolean;
  requireSpecial?: boolean;
}
```

15. Advanced Patterns

Server-side pagination with osl-setup

```
async onPageChange(e: OslPageEvent) {
  this.currentPage = e.page;
  await this.loadData();
}

async onPageSizeChange(size: number) {
  this.pageSize = size;
  this.currentPage = 1;
  await this.loadData();
}

async onSortChange(e: OslSortEvent) {
  this.sortKey = e.key;
  this.sortDir = e.direction;
  await this.loadData();
}

private async loadData() {
  this.loading = true;
  const res = await this.svc.search({
    page: this.currentPage,
    pageSize: this.pageSize,
    sortKey: this.sortKey,
    sortDir: this.sortDir,
  });
}
```

```

    query:    this.searchQuery,
  });
  this.loading = false;
  if (res.isSuccessful) {
    this.datasource = res.result;
    this.totalRecords = Number(res.headers?.get('recordCount') ?? 0);
  }
}

```

Form with cascading dropdowns

```

formElements: elements[] = [
  {
    key: 'countryId',
    label: 'Country',
    elementType: 'select',
    columns: 6,
    datasource: this.countries,
    displayField: 'name',
    valueField: 'id',
    change: (model) => {
      // Clear city when country changes
      model.cityId = null;
      // Reload cities
      this.loadCities(model.countryId);
    },
  },
  {
    key: 'cityId',
    label: 'City',
    elementType: 'select',
    columns: 6,
    datasource: this.cities,           // update this array on change above
    displayField: 'name',
    valueField: 'id',
    disabledIf: () => !this.formModel.countryId,
  },
];

```

Multiple services — domain separation

```

// auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService extends Httpbase {
  constructor() { super('Auth'); }
  async login(body: LoginDto) { return this.post<TokenDto>('Login', body); }
  async logout() { return this.post<void>('Logout', {}); }
  async refreshToken() { return this.post<TokenDto>('Refresh', {}); }
}

```

```
// user.service.ts
@Injectable({ providedIn: 'root' })
export class UserService extends Httpbase {
  constructor() { super('User'); }
  // getAll, getById, save, update, remove all inherited
  async changePassword(dto: ChangePasswordDto) {
    return this.patch<void>('ChangePassword', dto);
  }
}

// report.service.ts
@Injectable({ providedIn: 'root' })
export class ReportService extends Httpbase {
  constructor() { super('Report'); }
  async export(params: ExportParams) {
    return this.post<Blob>('Export', params);
  }
}
```

File upload with progress tracking

```
async uploadDocument(file: File) {
  const fd = new FormData();
  fd.append('file', file);
  fd.append('entityId', String(this.entityId));

  const res = await this.docSvc.upload<{ id: number; url: string }>('Upload', fd);

  if (res.isSuccessful) {
    this.base.showSuccess('Document uploaded successfully.');
```

```
    this.attachments.push(res.result);
  } else {
    this.base.showError(res.error);
  }
}
```

Skeleton with reactive theme switching

```
@Component({
  template: `
    <button (click)="toggleDark()">Toggle Dark Mode</button>

    <div [oslSkeleton]="loading"
      oslSkeletonType="table"
      [oslSkeletonTableRows]="10">
      <table>...</table>
    </div>
  `
})
```

```
})  
export class DashboardComponent {  
  private skeletonTheme = inject(OslSkeletonThemeService);  
  isDark = false;  
  
  toggleDark() {  
    this.isDark = !this.isDark;  
    this.skeletonTheme.setTheme(this.isDark ? 'dark' : 'light');  
  }  
}
```

16. Troubleshooting

NullInjectorError: No provider for HttpClient

Add `provideHttpClient()` to your app config:

```
// app.config.ts  
providers: [provideHttpClient()]
```

Dialog components are not rendering

Ensure your feature components are declared in the same module as `FormStructureModule`, or are standalone components.

Skeleton does not appear on a flex/grid child

Set a minimum height on the host element:

```
<div [oslSkeleton]="loading" oslSkeletonMinHeight="200px"></div>
```

API datasource in form loads on every dialog open

`DatasourceCacheService` caches per `(service, method, body)` key. If the datasource is not being cached, verify the `apiService` reference is the same injected instance (not a new object) every time the form opens.

`beforeDisplay` not called on edit

`beforeDisplay` is only called when opening the edit dialog. Pass a pure function — it receives the row object and must return the transformed model:

```
beforeDisplay = (row: Product) => ({  
  ...row,
```

```
    price: row.price * 100, // convert cents to display value
  });
```

JWT token not sent on requests

The service reads `localStorage.getItem('token')`. Verify the key name matches exactly — it is case-sensitive.

osl-base-extended — © Bilal Raza — ISC License

To convert this file to PDF: use VS Code "Markdown PDF" extension, *md-to-pdf DOCS.md*, or Pandoc.